

Exercícios Complementares - Semana 03

Universidade Tecnológica Federal do Paraná (UTFPR), campus Apucarana
Curso de Engenharia de Computação
Disciplina de Programação Orientada a Objetos - POCO4A
Prof. Dr. Rafael Gomes Mantovani

1. Crie uma classe **Livro** com atributos título, autor e ano. Implemente construtores que aceitem diferentes combinações de parâmetros (com e sem ano). Em **C++**, use o destrutor para exibir uma mensagem de desalocação. Em **Python**, implemente `__del__` e `__init__`. O que deve ser feito:

- Construtor padrão e construtor com parâmetros
 - Em **C++**: destrutor que imprime mensagem ao sair de escopo
 - Em **Python**: `__del__` equivalente
 - Criar um vetor/lista de livros e observar a ordem de destruição
-

2. Implemente uma classe **Sensor** que controla quantas instâncias existem ao mesmo tempo. O construtor incrementa um contador estático; o destrutor o decrementa. O programa deve mostrar a contagem a cada criação e destruição. O que deve ser feito:

- Atributo estático compartilhado entre instâncias
 - Construtor incrementa; destrutor decrementa
 - Testar dentro de escopos aninhados para ver o comportamento
 - Em **Python**: usar variável de classe e `__del__`
-

3. Implemente uma lista encadeada simples (sem usar `std::list` ou `list()`). A classe **No** deve liberar o próximo nó em seu destrutor. A classe **Lista** deve ter construtor, construtor de cópia e destrutor. Analise o risco de *stack overflow* em listas grandes. O que deve ser feito:

- No com destrutor que deleta próximo (*delete next*)
 - Construtor de cópia que realiza *deep copy*
 - Destruitor da **Lista** não pode vaziar memória
-

4. Crie um **PoolMemoria** em que pré-aloca um bloco fixo e usa *placement new* para construir objetos dentro dele. O destrutor do pool deve chamar explicitamente os destrutores de todos os objetos antes de liberar o bloco.

- Pré-alocação com *new char[N]*
 - *Placement new* para construir objetos no pool
 - Chamada explícita de destrutor: `obj-> Tipo()`
 - Destruitor do pool libera o bloco inteiro ao final
-

5. Crie uma classe **Processo** que, ao ser construída, se registra em um gerenciador global (classe **Gerenciador** com lista estática). Ao ser destruída, remove a si mesma do registro. O **Gerenciador** expõe um método que lista todos os processos vivos. O que deve ser feito:

- Construtor insere ponteiro/referência no **Gerenciador**
- Destruitor remove a entrada correspondente
- **Gerenciador** tem método `listar_ativos()`
- Testar criação em escopos diferentes para ver entradas e saídas